

P / G → Q

CARRIER SET ANALYSIS & CURRICULUM ARCHITECTURE

A Complete Teaching Curriculum & Implementation Manual for Propagation Logic

James Pugmire | Propagation Logic Project · 2026

Tools: `carrier_tool.py` | `CarrierExplorer.jsx` | `pl_unified.py`

OVERVIEW: THE THREE QUESTIONS

A formal system is completely specified by three parameters. This is not an approximation or a simplification. It is a precise operational claim: provide the architecture with **V**, **Γ**, and **θ**, and the internal environment will map out exactly what laws the system forces—without importing external or extraneous axiomatic structures from outside.

Parameter Matrix Component	Description & Domain Mappings
1. V — Value Carrier	The designation value space. Examples: <code>{0,1}</code> , <code>{0,B,1}</code> , <code>[0,1]</code> , $\mathbb{R}$, $\mathbb{N}$, $\mathbb{C}$
2. Γ — Gradient Family	The set of structurally available operational fields. Examples: <code>full_bool</code> , <code>constructive</code> , <code>linear</code> , <code>differential</code> , <code>measure</code>
3. θ — Coherence Threshold	The metric that establishes when a pattern achieves equilibrium. Examples: <code>1.0</code> (boolean standard), <code>0.0</code> (infinitesimal), <code>2.0</code> (relaxed)

Operational Invariant Constraint

P / G → Q

Change V, Γ, or θ. Watch the forced laws change. The mechanism is constant. Only the parameters change.

What This Is Not

This paradigm is not a reductive claim that all formal systems are essentially identical. They are fundamentally distinct operations. Classical logic and intuitionistic logic force drastically separate behaviors. The point is that those core differences are direct consequences of exactly one parameter modification (**Γ**), not of mysterious independent metaphysical commitments.

Furthermore, this is not an argument that traditional axiom systems are invalid; axioms are precise definitions within their respective domains. The core thesis asserts that every axiom represents a static boundary condition of a specific propagation regime—mechanically forced, not freely chosen.

MODULE 1: THE MECHANISM

Duration: 10 minutes. **Prerequisite:** None.

1.1 The Single Operator

Everything within the domain of Propagation Logic builds from a single atomic primitive operator:

$$P /_C G := Q$$

A loaded pattern **P** propagates through context **C** under the constraints of a gradient field **G** to generate output state **Q**. This is the complete operational primitive.

Definition: Loaded Pattern $P = (v, L)$

v (Designation Component): The specific evaluation value contained within the designated value carrier **V**. (e.g., 0 or 1 in boolean spaces; 0.7 in probability parameters; 3.14 in real values).

L (Informational Load): The scalar magnitude tracking accumulated propagation history ($|H_P| \geq 0$). $L = 0$ represents a seed state with no historical overhead. $L > 0$ indicates structural history in active motion.

CRITICAL IMPERATIVE: **v** and **L** are distinct variables. Conflating them is the root error. A pattern can be designated ($v=1$) but incoherent ($L > \theta$). Conversely, it can be coherent ($L \leq \theta$) but completely undesignated ($v=0$). Systematic validity strictly requires both conditions: $v=1$ AND $L \leq \theta$.

Definition: Context $C = (\Gamma, \theta)$

Γ (Gradient Family): The comprehensive domain of available operational gradient fields.

θ (Coherence Threshold): The metric setting the limits of stability. When internal demand drops to zero, the pattern state settles into equilibrium.

$$\begin{aligned} \text{demand}(P, C) &= \max(0, L - \theta) \\ \text{rate}(P, C) &= \min(1, \theta/L) \text{ for } L > 0; \quad \text{rate} = 1 \text{ for } L = 0 \end{aligned}$$

Coherence Status: Pattern **P** is coherent in **C** iff $\text{demand}(P, C) = 0$ (meaning $L \leq \theta$). Pattern **P** achieves validity in **C** iff $v = 1$ AND it is fully coherent.

1.2 The Propagation Ordering

Theorem (Propagation Ordering): Among structurally incoherent patterns where ($L > \theta$):

$$\text{rate}(P, C) > \text{rate}(Q, C) \iff L_P < L_Q$$

Proof: Since rate evaluates to θ/L for all active incoherent tracks, $\theta/L_P > \theta/L_Q$ holds true if and only if $L_P < L_Q$ given a positive threshold ($\theta > 0$). $\square$

Consequence: Structurally simpler patterns propagate faster through spatial fields. Excess complexity generates downstream computational drag. Systematic reconfigurations consistently shift toward states

optimizing for lower historical load. This fundamental ordering directly governs biological evolution, cognitive organization paradigms, Newton's method, and deep machine learning gradient descent steps.

1.3 Exercises

Exercise E1.1

Given a pattern state $P = (v=1, L=1.6)$ evaluating within a context C defined by $\theta=1.0$: Calculate the absolute demand, current propagation rate, and determine its structural classification (coherent, valid, or neither).

Exercise E1.2

Evaluate a baseline seed state defined as $P = (v=0, L=0)$. Compute its corresponding demand value and active execution rate. Is the pattern structurally coherent?

Exercise E1.3

Analyze two competing patterns moving through a field where $\theta=1.0$: $P_1 = (1, 0.8)$ and $P_2 = (1, 2.1)$. Identify which pattern propagates faster toward systematic coherence, and explicitly compute the exact rate delta.

MODULE 2: THE CARRIER SETS THE LOGIC

Duration: 20 minutes. **Core Concept:** Modify V to alter forced systemic laws.

2.1 Classical Logic from $V = \{0,1\}$

When the underlying architecture selects the discrete binary value space $V = \{0,1\}$ alongside a complete boolean gradient family and standard stability boundaries ($\theta = 1.0$), the local environment forces specific identities:

Derivation Map: Law of Non-Contradiction (LNC)

For all baseline structural assignments within the set: $v \cdot (1 - v) = 0$.

$$G_and(P, G_neg(P)) = G_and((v, L), (1-v, L)) = (v \cdot (1-v), 2L) = (0, 2L)$$

Because the evaluation arithmetic yields identically zero across the entire domain of $V=\{0,1\}$, the Law of Non-Contradiction is revealed as a derived state transition constraint rather than a freely chosen axiom. Shift V to a multi-valued topology, and LNC instantly uncouples.

Derivation Map: Law of Excluded Middle (LEM)

For all elements within the target set: $\max(v, 1 - v) = 1$.

$$G_{\text{or}}(P, G_{\text{neg}}(P)) = (\max(v, 1-v), \min(L, L)) = (1, L)$$

Because one component of any complementary value pair must evaluate to unity inside this specific space, the Law of Excluded Middle is strictly forced by the geometry of $V=\{0,1\}$. If Γ shifts to a constructive family, LEM falls away as unreachable due to proof path constraints, even though value arithmetic remains static.

2.2 Parameter Covariation Analysis

The parameter matrix tracks structural changes cleanly: modifying exactly one architectural property shifts specific systematic laws while the baseline processing engine remains constant.

Initial System State	Target Parameter Shift	Resulting Formal Engine	Forced Law Vector Mutations
Classical	Γ : full $\rightarrow$ constructive	Intuitionistic Logic	LEM & DNE transform from forced to unreachable.
Classical	V : $\{0,1\} \rightarrow \{0,B,1\}$	Paraconsistent (LP)	LNC, LEM, and Ex Falso parameters structural collapse.
Classical	Γ : full $\rightarrow$ linear	Linear Logic	Resource sensitivity transforms into a forced operation.
Classical	Γ : full $\rightarrow$ relevance	Relevance Logic	Strict parameter historical overlap requirements enforced.
Classical	V : $\{0,1\} \rightarrow \mathbb{R}$, Γ : differential	Differential Calculus	Forces Leibniz rule and the Fundamental Theorem of Calculus.
Classical	V : $\{0,1\} \rightarrow [0,1]$, Γ : measure	Probability Theory	Forces strict conformance to Kolmogorov probability axioms.

2.3 Running the Tool

```
# Executing explicit system mapping via the reference engine command line:
python carrier_tool.py classical           # Compiles a complete forced law report
python carrier_tool.py --learn             # Initialized the interactive pedagogical guide
python carrier_tool.py --diff classical intuitionistic # Isolate exact structural updates
python carrier_tool.py --compare classical paraconsistent

# Utilizing the framework directly as an embedded automation library:
from carrier_tool import CarrierSet
cs = CarrierSet.preset('classical')
print(cs.forces('lnc')) # Returns True
print(cs.forces('lem')) # Returns True

# Reconfiguring the parameter space programmatically to monitor state mutations:
cs2 = CarrierSet(V='{0,1}', Gamma='constructive', theta=1.0)
print(cs2.forces('lem')) # Returns False - LEM becomes unreachable under constructive Γ restricti
print(cs2.forces('lnc')) # Returns True - LNC remains strictly forced by baseline carrier arithm
```

2.4 Exercises

Exercise E2.1

Isolate the single minimal parameter adjustment required to transition a formal classical engine into a paraconsistent (LP) state. Catalog every law that changes state during this transition.

Exercise E2.2

Design a precise custom value space where the Law of Non-Contradiction holds completely true, yet the Law of Excluded Middle fails. Detail the explicit settings for the $(\mathbf{V}, \mathbf{I}, \mathbf{\theta})$ parameter vector.

Exercise E2.3

Identify the exact parameter alterations within the context graph configuration that differentiate Modal S4 from Modal S5 architectures.

Exercise E2.4

Map out the parameter configuration for a standard fuzzy logic execution layout. Define the exact bounds of the target value carrier space $\mathbf{V}$.

MODULE 3: THE REFERENCE TABLE

Duration: 15 minutes. **Core Objective:** Complete mastery of the comprehensive three-parameter axis space.

3.1 The Comprehensive Parameter Matrix

Every row in this matrix is a formal system specified by three parameters. Read the matrix as: *"Given the configuration vector $(\mathbf{V}, \mathbf{I}, \mathbf{\theta})$, the listed structural laws are mechanically forced by the space."*

System Classification	V Space	Γ Family	θ Bound	LNC	LEM	Ex Falso	Leibniz	FTC
Classical Logic	{0,1}	full_bool	1.0	✓	✓	✓	✗	✗
Intuitionistic Logic	{0,1}	constructive	1.0	✓	✗	✓	✗	✗
Paraconsistent (LP)	{0,B,1}	full	1.0	✗	✗	✗	✗	✗
Linear Logic	{0,1}	linear	1.0	✓	✓	✓	✗	✗
Relevance Logic	{0,1}	relevance	1.0	✓	✓	✓	✗	✗
Modal K / S4 / S5	{0,1}	modal (graph variant)	1.0	✓	✓	✓	✗	✗
Probability Theory	[0,1]	measure	0.0	—	—	—	✗	✗
Calculus Framework	$\mathbb{R}$	differential	0.0	—	—	—	✓	✓
Peano Arithmetic	$\mathbb{N}$	successor	0.0	—	—	—	✗	✗

✓ = Strict structural law forced by the geometry of the value space. ✗ = Not forced by the carrier. — = Law not applicable within this specific carrier type domain.

3.2 Structural Cross-Carrier Correspondences

Isomorphic operational frameworks express identical structures across vastly distinct underlying value parameters.

Discrete Logic Domain: $V = \{0,1\}$	Continuous Field Domain: $V = \mathbb{R}$	Isomorphic Structural Core Shared
$\neg\neg P = P$	$d/dx \int f \, dx = f \quad (\text{FTC})$	Inverse operations achieve perfect identity cancelation: $G^{-1} \circ G = G_{id}$
Gödel Boundary: $\text{load} \rightarrow \infty$	Divergent Series: $\text{load} \rightarrow \infty$	System reaches localized limit capacity boundaries.
System Tautologies	$\exp(x), \sin(x) / \cos(x)$	Invariant operational balance configurations under spatial progression.
Excluded Middle ($v \vee \neg v = 1$)	Mean Value Theorem	Enforces mathematical completeness across the active field space.
LNC Variant Identity ($v \cdot (1-v) = 0$)	Integration by Parts	Exposes clear bilinear partition and distribution properties.

3.3 Exercises

Exercise E3.1

State precisely which topological property within the accessibility graph configuration separates a Modal K architecture from a Modal S4 platform.

Exercise E3.2

Explain explicitly why a continuous probability value space ($V = [0,1]$) does not force basic structural logic identities like LNC or LEM. Define the structural meaning of "both" within a continuous measure carrier.

Exercise E3.3

The Fundamental Theorem of Calculus exhibits the identical structural composition as Double Negation Elimination. State the precise invariant property shared between them and why it emerges uniformly across both environments.

MODULE 4: PARADOX PROFILES

Duration: 15 minutes. **Core Strategy:** Replace abstract metaphysical mystery with precise load profile diagnostics.

4.1 Structural Failure Mode Breakdown

Within the Propagation Logic framework, standard logical paradoxes do not exist as unsolvable contradictions; they represent clear execution anomalies where a pattern's gradient demand expands beyond the localized context coherence limit (θ). They map into three distinct structural profiles:

TYPE 1 – Value Self-Reference (Example: The Liar Paradox)

The equation $v = f(v)$ possesses zero invariant balance states or fixed points within the target set V . Crucially, the internal informational load L does not diverge toward infinity; the tracking values enter a state of endless oscillation. This lack of a fixed point is proven via finite exhaustive verification inside the set boundary.

TYPE 2 – History Self-Reference (Examples: Gödel Incompleteness / Turing Halting)

The historical matrix overlaps natively upon its own footprint: $H_P \supseteq H_P$. Internal load diverges exponentially according to execution step depth: $L(d) = 2^d$. This reveals that the Halting Problem and Gödelian Incompleteness share an identical geometric breakdown signature, where resolving the state requires a non-finite contextual threshold.

TYPE 3 – Gradient-Pattern Cut Collapse (Example: Russell's Paradox)

The governing membership operational gradient attempts to rule upon its own baseline execution scope at the same operational tier. Load values remain bounded, but the structural separation between operational rules and base elements collapses. Resolving this constraint requires explicit relational context stratification.

4.2 Mapping the Profiles in Code

```
# TYPE 1: Bounded Value Self-Reference Execution Profile
liar = lambda v: 1 - v
fixed_point = next((v for v in [0,1] if liar(v) == v), None)
assert fixed_point is None # PROVED: Bounded value space contains zero fixed points.

# TYPE 2: Divergent History Self-Reference Tracking
informational_load = 1.0
for depth in range(8):
    informational_load *= 2 # Deep simulation loops compound context size exponentially
    # Step Progression: d=0 -> L=2, d=3 -> L=16, d=6 -> L=128. Bounded thresholds fail.

# TYPE 3: Structural Cut Collapse Diagnosis
# The gradient defines the valid context tier.
# Constraint mapping: L_gradient > 0_C iff L_gradient > L_gradient -> Contradiction.
# Resolution requires strict operational stratification. Type Theory handles this balance.
```

4.3 Yablo's Sequence — Infinite Construction Bypassing Self-Reference

Yablo's paradox (S_n : asserting that every subsequent statement S_k where $k > n$ is false) contains zero expressions referencing themselves. Yet it crashes the processing layer. In this framework, the load profile diagnostics reveal why: building state S_1 requires evaluating the structural assignment of S_2 , which immediately requires evaluating S_3 , continuing infinitely. There is no valid zero-load seed state ($L=0$) to begin processing.

$$\text{Total System Construction Cost} = \sum_{n=1}^{\infty} \text{Landauer}(300K) \rightarrow \infty$$

This is not an abstract logical loop; it is a structural resource violation: attempting to draw an infinite overdraft on a balance sheet with no initial capital deposit. The pattern is broken before execution can begin.

4.4 Exercises

Exercise E4.1

Classify the precise structural failure profile (Type 1, Type 2, or Type 3) for: (a) The Barber Paradox, (b) Berry's Paradox, and (c) The Banach-Tarski Theorem anomaly.

Exercise E4.2

Type 2 history loops force an exponential processing load of $L_d = 2^d$. Analyze what this mathematical curve predicts regarding the absolute physical memory requirements for an artificial intelligence system running an uncensored, mathematically complete internal model of its own operations. Is this an engineering challenge or a fundamental constraint?

Exercise E4.3

Explain clearly why Type 3 (Russell) execution failures demand strict parameter hierarchy context stratification instead of simply scaling up the local coherence threshold value θ .

MODULE 5: EXTENDING THE FRAMEWORK

Duration: 30 minutes. **Objective:** Standard protocol for onboarding new formal systems.

5.1 The New Carrier Onboarding Protocol

Every formal system interfaces with this environment by translating its operational constraints into a clean three-parameter vector:

1. **Isolate Value Space V :** Explicitly declare the precise topology of designation values. Determine if the values are discrete binary elements, continuous intervals, linear matrices, or networked propositional states.
2. **Isolate Gradient Family Γ :** Map the explicit operations driving data modification. Define how negation behaves, how conjunction intersections compress information, and cleanly separate derived operations from base primitives.
3. **Isolate Coherence Threshold θ :** Calculate the metric marking complete systemic stability. Define exactly what condition maps to a zero-demand state (demand = 0).
4. **Derive Forced Properties:** Do not pre-authenticate or assume standard identities. Test classical law vectors against the system's baseline arithmetic to see if they are forced or imported from outside. If imported, explicitly label them as axioms.
5. **Execute and Verify:** Export the new definition block directly to JSON via the reference package tools for automated community processing: `carrier_tool.py --export custom_system > custom.json`.

5.2 Community Research Challenges

Challenge Matrix A: Łukasiewicz Fuzzy Logic Specification

Value Space Assignment: Continuous real intervals: $V = [0,1]$.

Gradient Primitives: $\neg A = 1 - A \mid A \wedge B = \max(0, A + B - 1) \mid A \vee B = \min(1, A + B)$.

Research Prompts: 1. Define the stability point threshold θ . 2. Derive if the system forces LNC arithmetic. 3. Derive if LEM remains valid. 4. Contrast the execution profile against paraconsistent systems.

Challenge Matrix B: Temporal Progression Systems (LTL / CTL)

Structural Proposition Mapping: Assertions map validation states across discrete temporal steps rather than static global frames.

Research Prompts: 1. Evaluate if $V=\{0,1\}$ suffices, or if the system requires a sequence mapping $V=\{0,1\}^{\mathbb{N}}$. 2. Determine the exact accessibility graph geometry. 3. Isolate which modal axioms are forced. 4. Define what θ tracks within a temporal context.

5.3 Serialization & Community Integration Specifications

```
# Instantiating and verifying a new custom formal carrier structure:
from carrier_tool import CarrierSet

my_custom_engine = CarrierSet(
    V='your_custom_value_space',
    Gamma='your_gradient_family_operations',
    theta=1.0,
    name='Validated Community Logic System',
    notes='Isolates and solves tracking failures across distributed execution contexts.'
)

# Exporting parameters for cross-platform network distribution:
print(my_custom_engine.to_json())
```

All structured logical parameter files are integrated directly into the centralized community platform folder at github.com/ApplePiesFromScratch/propagation-logic/carriers/.

5.4 The Falsifiability Metric Standard

Every programmatic system submission to the community registry must satisfy a three-part validation criteria:

- **Derivable Forced Vector:** Isolate at least one structural law that is derived from value space arithmetic.
- **Explicit Boundary Limit:** Document at least one structural identity that explicitly fails validation, proving the system's absolute boundaries.
- **Computational Falsification Vector:** Provide a precise, automated test routine capable of proving the system definition incorrect if its underlying arithmetic models break down.

CORE QUICK-REFERENCE INDEX

Operation / Metric	Governing Mathematical Formula	Primary Value Space	System Execution Profiles & Operational Status
Non-Contradiction	$v \cdot (1 - v) = 0$	$\{0,1\}$	Forced arithmetic identity. Not an independent axiom.
Excluded Middle	$\max(v, 1 - v) = 1$	$\{0,1\}$	Forced binary identity. Unreachable in constructive paths.
Double Negation	$1 - (1 - v) = v$	$\{0,1\}$	Involution behavior. Inference fails inside constructive frameworks.
Gradient Demand	$\max(0, L - \theta)$	All Containers	Tracks the volume of informational history a context cannot absorb.
Propagation Rate	$\min(1, \theta/L)$	All Containers	Enforces the baseline priority rule: simpler patterns propagate faster.
Leibniz Product Rule	$d(f \cdot g) = f \cdot dg + g \cdot df$	$\mathbb{R}$ Space	Strictly forced by spatial boundary conditions where $\epsilon^2 = 0$.
Fundamental Calculus	$G_{diff} \circ G_{int} = G_{id}$	$\mathbb{R}$ Space	Opposite gradients cancel, isomorphic to double negation.
Kolmogorov Axioms	$P(\Omega)=1, P(A \cup B)=P(A) + P(B)$	$[0,1]$ Space	Mechanically forced by normalized measures over valid contexts.
Liar Load Profile	Infinite Bounded Oscillation	$\{0,1\}$	Type 1 value failure: the environment contains zero fixed points.
Gödel Load Profile	$L(d) = 2^d$	PA / TM Spaces	Type 2 history loop: informational overhead expands exponentially.
Russell Structure	Cut-Collapse Crash	Set Spaces	Type 3 structural error: demands strict contextual stratification.

REFERENCES & VERIFICATION TOOLS

carrier_tool.py: Native command-line verification and diagnostic comparison suite. Execute via: `python carrier_tool.py --learn`.

CarrierExplorer.jsx: Front-end React layout framework for rendering interactive parameter spaces across web interfaces.

pl_unified.py: Central execution repository containing deep algorithmic validations for all components across Sections 1 to 12.

• Pugmire, J. (2026). *Propagation Logic v13 Architecture Spec*. Repository tracking: github.com/ApplePiesFromScratch/propagation-logic

• Pugmire, J. (2026). *DRAS: De-Reification Axiom Standard Manual*.

